

Proyecto Final

BitQuest: Explorador de Matrices con C y NASM

Lenguaje Ensamblador
Código híbrido C + NASM de 64 bits

Fecha de entrega: 15 de junio del 2026

Descripción general del proyecto

El proyecto final consiste en desarrollar un videojuego en consola utilizando programación híbrida con **C y NASM de 64 bits**. El juego llevará por nombre:

BitQuest: Explorador de Matrices con C y NASM

El videojuego deberá representar un laberinto mediante una matriz grande. El jugador se moverá dentro del mapa, recolectará monedas, buscará una llave, abrirá una puerta y llegará a una salida para avanzar de nivel.

A diferencia de un juego donde se muestra todo el mapa en pantalla, en este proyecto el mapa completo será más grande que la zona visible. Por ejemplo, cada nivel podrá tener una matriz de **60 x 60**, pero en pantalla solo se mostrará una ventana de **20 x 20**. Conforme el jugador se mueva, la vista deberá desplazarse para mostrar la zona cercana al jugador.

El objetivo principal es reforzar el uso de matrices, funciones, paso de parámetros, compilación híbrida, manejo de memoria, ciclos, condiciones y llamadas a rutinas escritas en ensamblador desde un programa en C.

Objetivo del proyecto

Desarrollar un videojuego en consola que integre código en C con rutinas escritas en NASM de 64 bits. El programa deberá demostrar que el estudiante comprende cómo representar información mediante matrices, cómo recorrer memoria, cómo validar movimientos dentro de un mapa y cómo llamar funciones en ensamblador desde un programa escrito en C.

Al finalizar el proyecto, el estudiante deberá ser capaz de:

- Representar un mapa mediante una matriz de caracteres.
- Mostrar únicamente una parte visible de una matriz más grande.
- Controlar el movimiento de un jugador dentro de un laberinto.
- Validar paredes, puertas, llaves, monedas y salidas.
- Usar funciones escritas en NASM llamadas desde C.
- Pasar parámetros desde C hacia NASM.
- Devolver resultados desde NASM hacia C.
- Compilar un proyecto híbrido usando NASM y GCC de 64 bits.

Descripción del juego

El juego estará compuesto por al menos **3 niveles**. Cada nivel tendrá un mapa representado por una matriz de caracteres. El jugador deberá recorrer el mapa, recolectar monedas, encontrar una llave, abrir una puerta y llegar a la salida.

Cada mapa completo deberá ser de al menos:

60 filas x 60 columnas

Sin embargo, en pantalla solo deberá mostrarse una ventana de:

20 filas x 20 columnas

Esto significa que el mapa completo existe en memoria, pero el usuario solo verá una parte del mismo. La vista visible deberá actualizarse conforme el jugador avance.

No es obligatorio que el jugador recolecte todas las monedas para terminar un nivel. Sin embargo, al finalizar cada nivel sí deberá mostrarse cuántas monedas recolectó respecto al total de monedas disponibles en ese nivel.

Elementos del mapa

El mapa deberá construirse usando caracteres simples para evitar problemas de compatibilidad entre terminales.

Símbolo	Significado
#	Pared. El jugador no puede atravesarla.
.	Camino libre. El jugador puede caminar sobre esta celda.
P	Jugador. Representa la posición actual del personaje.
M	Moneda. El jugador puede recolectarla.
K	Llave. Permite abrir la puerta del nivel.
D	Puerta. Solo puede cruzarse si el jugador tiene la llave.
E	Salida. Permite terminar el nivel.

Nota

Pueden utilizar otros caracteres visuales, colores o bloques ASCII como mejora opcional. Sin embargo, la versión base debe funcionar correctamente usando caracteres simples.

Ejemplo visual del mapa

El mapa completo podría ser una matriz grande de 60 x 60. En el siguiente ejemplo se muestra solo una versión reducida para ilustrar la idea:

```
#####
#P....#.....#.....#.....#
#...#.#.#####.#.#####. .#.#.#####.#
#...#.#.....M.....#.....M.....#.....M.....#
###.#.#####.#####.#####.#####
#...#.....K.....#.....D....#
#.#.#####.#####.#####.#####.#
#.....#.....E#
#####
```

Sin embargo, el jugador no deberá ver todo el mapa al mismo tiempo. En pantalla solo deberá mostrarse una ventana de 20 x 20.

Ejemplo de vista visible:

```
#####
#P....#.....#
#...#...#####.#
#...#...#.....M....#
###.#...#####.#####
#...#.....#
#.....#####
#.....#
#####
```

Nota

Los ejemplos anteriores son únicamente ilustrativos. Los mapas reales del proyecto deberán tener al menos 60 filas y 60 columnas.

Movimiento del jugador

El jugador deberá moverse usando las siguientes teclas:

Tecla	Acción
W o w	Mover hacia arriba.
A o a	Mover hacia la izquierda.
S o s	Mover hacia abajo.
D o d	Mover hacia la derecha.
Q o q	Salir del juego.

El programa deberá validar que el jugador no atravesase paredes. Si el jugador intenta moverse hacia una pared, su posición no deberá cambiar.

Reglas del juego

El juego deberá cumplir con las siguientes reglas:

1. El jugador inicia en una posición marcada con el carácter P.
2. El jugador puede moverse por los caminos libres representados con ..
3. El jugador no puede atravesar paredes representadas con #.

4. El jugador puede recolectar monedas representadas con M.
5. Cada moneda recolectada deberá aumentar el contador de monedas del jugador.
6. Después de recolectar una moneda, la celda correspondiente deberá convertirse en camino libre.
7. El jugador puede recoger una llave representada con K.
8. La puerta representada con D solo podrá abrirse si el jugador ya tiene la llave.
9. Si el jugador no tiene llave, no podrá cruzar la puerta.
10. Si el jugador tiene llave, podrá cruzar la puerta.
11. El jugador termina el nivel al llegar a la salida representada con E.
12. Al terminar cada nivel, el programa deberá mostrar cuántas monedas recolectó el jugador respecto al total de monedas del nivel.
13. Al terminar los tres niveles, el programa deberá mostrar un resumen final.

Requisito de niveles

El proyecto deberá incluir al menos **3 niveles obligatorios**.

Nivel	Descripción esperada
Nivel 1	Laberinto sencillo para familiarizar al usuario con la mecánica.
Nivel 2	Laberinto intermedio con más caminos y monedas.
Nivel 3	Laberinto más amplio o con una distribución más retadora.

Cada nivel deberá tener su propio mapa. Los tres mapas deberán cumplir con el tamaño mínimo de **60 x 60**.

Nota

Los mapas pueden estar declarados directamente en archivos de C o en archivos de encabezado. No es obligatorio leer los mapas desde archivos externos, aunque esto puede considerarse como mejora opcional.

Ventana visible de 20 x 20

Aunque el mapa completo sea de 60 x 60, el programa solo deberá mostrar una parte de 20 x 20. Esta ventana visible deberá moverse conforme el jugador avance por el mapa.

La ventana visible deberá calcularse a partir de la posición actual del jugador. La intención es que el jugador vea únicamente una zona cercana a su ubicación, no el mapa completo.

Para cumplir este requisito, el programa deberá considerar lo siguiente:

- La posición actual del jugador dentro del mapa completo.
- El tamaño total del mapa.
- El tamaño de la ventana visible.
- Los límites superiores, inferiores, izquierdos y derechos del mapa.

Si el jugador se encuentra cerca de los bordes del mapa, la ventana visible deberá ajustarse para no intentar imprimir posiciones fuera de la matriz.

Importante

No se permite mostrar el mapa completo en pantalla como sustituto de la ventana visible. El proyecto debe demostrar que el estudiante puede trabajar con una matriz grande y mostrar solo una parte de ella.

Nota

No es obligatorio que la ventana esté perfectamente centrada todo el tiempo, pero sí debe cumplirse que solo se muestre una parte del mapa y que la vista avance conforme el jugador se desplaza.

Uso obligatorio de NASM

El proyecto deberá incluir **cinco funciones obligatorias implementadas en NASM** y llamadas desde C.

No se aceptará un proyecto donde todo esté programado únicamente en C. Tampoco se aceptará un proyecto donde NASM solo aparezca de forma decorativa, es decir, sin participar realmente en la lógica del programa.

Importante

Todas las funciones descritas en esta sección son obligatorias. No se consideran funciones opcionales de NASM.

Función obligatoria 1: contar caracteres del mapa

La primera función obligatoria en NASM deberá contar cuántas veces aparece un determinado carácter dentro del mapa.

Esta función deberá recibir desde C:

- La dirección inicial del mapa.
- El número total de celdas.
- El carácter que se desea contar.

La función deberá recorrer la matriz en memoria y devolver cuántas veces aparece el carácter solicitado.

Esta función se utilizará principalmente para calcular cuántas monedas existen en cada nivel.

Importante

El total de monedas de cada nivel debe calcularse usando NASM al cargar o iniciar el nivel. No se permite que el alumno cuente manualmente las monedas del mapa y escriba directamente el número en una variable.

Función obligatoria 2: validar movimiento

La segunda función obligatoria en NASM deberá validar si una posición del mapa puede ser ocupada por el jugador.

Esta función deberá recibir desde C:

- La dirección inicial del mapa.
- El número de columnas del mapa.
- La nueva fila propuesta para el jugador.
- La nueva columna propuesta para el jugador.

La función deberá devolver:

Valor devuelto	Significado
1	El movimiento es válido.
0	El movimiento está bloqueado.

Como mínimo, esta función deberá validar que el jugador no se mueva hacia una pared representada con #.

Función obligatoria 3: calcular puntaje

La tercera función obligatoria en NASM deberá calcular el puntaje del jugador.

Esta función deberá recibir desde C los datos necesarios para calcular el puntaje, por ejemplo:

- Monedas recolectadas.
- Pasos realizados.
- Niveles completados.

El puntaje puede definirse con una fórmula sencilla. Por ejemplo, se puede dar mayor valor a las monedas recolectadas y restar puntos por la cantidad de pasos realizados.

El alumno deberá explicar en el reporte qué fórmula utilizó para calcular el puntaje.

Nota

La fórmula exacta del puntaje puede variar, pero el cálculo debe realizarse en NASM y el resultado debe mostrarse desde el programa principal.

Función obligatoria 4: detectar objeto en una celda

La cuarta función obligatoria en NASM deberá detectar si en una posición específica del mapa existe un determinado objeto.

Esta función deberá recibir desde C:

- La dirección inicial del mapa.
- El número de columnas del mapa.
- La fila que se desea revisar.
- La columna que se desea revisar.
- El carácter del objeto que se desea buscar.

La función deberá devolver:

Valor devuelto	Significado
1	El objeto buscado se encuentra en la celda indicada.
0	El objeto buscado no se encuentra en la celda indicada.

Esta función puede utilizarse para detectar monedas, llaves, puertas o salidas.

Función obligatoria 5: contar celdas libres

La quinta función obligatoria en NASM deberá contar cuántas celdas libres existen en el mapa.

Esta función deberá recibir desde C:

- La dirección inicial del mapa.
- El número total de celdas.

La función deberá recorrer el mapa y contar las celdas que representan camino libre.

Esta función tiene como propósito reforzar el recorrido de arreglos en memoria y permitir que el alumno practique ciclos, comparaciones y conteo de elementos desde NASM.

Nota

El resultado de esta función puede mostrarse al iniciar el nivel, en el resumen del nivel o en el reporte de ejecución. Lo importante es que la función exista, sea llamada desde C y devuelva un resultado real obtenido al recorrer la matriz.

Conteo de monedas

Cada nivel tendrá una cantidad de monedas colocadas dentro del mapa. El programa deberá mostrar al jugador cuántas monedas ha recolectado y cuántas monedas existen en total.

Durante el juego se debe mostrar algo similar a:

```
Nivel: 1
Llave: No
Pasos: 34
```

Al terminar un nivel, deberá mostrarse un resumen:

```
=====
Nivel 1 completado
Monedas recolectadas: 10 / 12
Pasos realizados: 85
=====
```

Al terminar los tres niveles, deberá mostrarse un resumen final:

```
=====
Juego completado
Monedas totales recolectadas: 34 / 41
Pasos totales: 278
Niveles completados: 3
Puntaje final: 3122
=====
```

Restricción importante sobre las monedas

No permitido

No está permitido contar manualmente las monedas del mapa y escribir el total directamente en el código.

Por ejemplo, no se aceptará que el alumno defina manualmente valores fijos para representar el total de monedas de cada nivel.

El total de monedas debe obtenerse mediante una función en NASM que recorra la matriz.

Importante

La función de NASM debe calcular el total de monedas a partir del contenido real del mapa. De esta forma, si se agregan o eliminan monedas del mapa, el total deberá actualizarse automáticamente sin modificar manualmente una variable.

Estructura sugerida del proyecto

Se recomienda organizar el proyecto en varios archivos para mantener el código limpio y comprensible.

```
BitQuest/
|
|-- main.c
|-- juego.c
|-- juego.h
|-- mapas.h
|-- rutinas.asm
|-- build.bat
|-- README.txt
```

Una posible distribución sería:

Archivo	Contenido sugerido
main.c	Función principal, menú inicial y control general del juego.
juego.c	Funciones del juego: imprimir mapa, mover jugador, cambiar nivel y mostrar resumen.
juego.h	Prototipos de funciones y constantes generales.
mapas.h	Declaración de los mapas de cada nivel.
rutinas.asm	Funciones implementadas en NASM.
build.bat	Archivo para compilar el proyecto en Windows.
README.txt	Instrucciones de compilación y ejecución.

Compilación esperada

El proyecto deberá compilarse usando NASM y GCC de 64 bits.

Para Windows, usando NASM y GCC de 64 bits, el proceso general puede ser:

```
nasm -f win64 rutinas.asm -o rutinas.obj
gcc main.c juego.c rutinas.obj -o BitQuest.exe
BitQuest.exe
```

En caso de usar Linux, el proceso general puede ser:

```
nasm -f elf64 rutinas.asm -o rutinas.o
gcc main.c juego.c rutinas.o -o BitQuest
./BitQuest
```

En caso de usar macOS, el proceso general puede cambiar por el formato de objetos y el nombre de los símbolos. Si el alumno trabaja en macOS, deberá documentar claramente los comandos usados para compilar y ejecutar.

Nota

Dependiendo del sistema operativo y del compilador utilizado, pueden existir diferencias en la convención de llamadas, nombres de símbolos y formato de objeto. El alumno deberá asegurarse de que su proyecto compile correctamente en el entorno indicado por el profesor.

Requisitos mínimos obligatorios

El proyecto deberá cumplir con los siguientes requisitos mínimos:

1. El proyecto debe estar programado usando C y NASM de 64 bits.

2. El juego debe ejecutarse en consola.
3. El juego debe tener al menos 3 niveles.
4. Cada nivel debe usar una matriz de al menos 60 x 60.
5. En pantalla solo debe mostrarse una ventana de 20 x 20.
6. El jugador debe moverse usando las teclas W, A, S, D.
7. El mapa debe incluir paredes, caminos, monedas, llave, puerta y salida.
8. El jugador no debe poder atravesar paredes.
9. El jugador debe poder recolectar monedas.
10. El jugador debe poder recoger una llave.
11. La puerta debe abrirse solo si el jugador tiene la llave.
12. El nivel debe terminar cuando el jugador llegue a la salida.
13. El total de monedas de cada nivel debe calcularse usando una función en NASM.
14. El programa debe mostrar monedas recolectadas contra monedas totales.
15. El programa debe mostrar un resumen al finalizar cada nivel.
16. El programa debe mostrar un resumen final al terminar los tres niveles.
17. El proyecto debe incluir las cinco funciones obligatorias implementadas en NASM.
18. El cálculo del puntaje debe realizarse mediante una función en NASM.
19. La detección de objetos en una celda debe realizarse mediante una función en NASM.
20. El conteo de celdas libres debe realizarse mediante una función en NASM.
21. El código debe estar organizado y comentado.
22. El proyecto debe entregarse con instrucciones claras de compilación y ejecución.

Aspectos no obligatorios

Para mantener una dificultad adecuada, los siguientes elementos no son obligatorios:

- Enemigos móviles.
- Trampas.

- Sistema de vidas.
- Movimiento aleatorio.
- Música o sonido.
- Gráficos externos.
- Interfaz gráfica.

Estos elementos pueden agregarse como mejoras adicionales, siempre que el proyecto base funcione correctamente.

Puntos extra

Los alumnos podrán obtener puntos extra si implementan mejoras adicionales al proyecto base.

Mejora	Puntos extra
Uso de colores en consola.	+5
Pantalla de inicio y pantalla de victoria.	+5
Lectura de mapas desde archivos <code>.txt</code> .	+5
Más de 3 niveles funcionales.	+5
Uso de bloques ASCII para mejorar la vista del mapa.	+5
Enemigo móvil sencillo con movimiento controlado.	+10

Importante

Los puntos extra solo se tomarán en cuenta si el proyecto base cumple correctamente con los requisitos mínimos.

Repositorio de GitHub

El código fuente del proyecto deberá estar alojado en un repositorio de GitHub. La entrega principal será un **archivo PDF**, y dentro de ese PDF deberá colocarse el enlace al repositorio donde se encuentra el código fuente completo.

El repositorio deberá contener, como mínimo:

- Los archivos fuente del proyecto: `.c`, `.h` y `.asm`.

- El archivo `build.bat` para compilar el proyecto en Windows.
- Un archivo `README.txt` o `README.md`.
- Los mapas, archivos auxiliares o recursos necesarios para ejecutar el juego, en caso de que existan.

Importante

El repositorio debe mostrar avance real del proyecto. No se aceptará como evidencia adecuada un repositorio con un solo commit realizado al final. Se espera observar varios commits que reflejen el desarrollo progresivo del proyecto.

El historial de commits deberá permitir observar que el proyecto fue construido por etapas. Por ejemplo, pueden existir commits relacionados con:

- Creación de la estructura inicial del proyecto.
- Implementación del mapa y la ventana visible.
- Movimiento del jugador.
- Integración de funciones en NASM.
- Conteo de monedas.
- Implementación de niveles.
- Correcciones y mejoras finales.

Nota

El archivo `README` no sustituye al reporte. Su propósito es indicar de forma breve cómo compilar y ejecutar el proyecto. Si el proyecto incluye un archivo `build.bat`, el `README` deberá explicar cómo utilizarlo para compilar o correr el juego.

Entregables

Entrega

La entrega será un **archivo PDF**. Dentro del PDF deberá incluirse el enlace al repositorio de GitHub donde se encuentra el código fuente completo del proyecto.

El alumno deberá entregar un PDF que contenga los siguientes elementos:

1. Portada del proyecto.
2. Enlace al repositorio de GitHub.
3. Capturas de pantalla de la ejecución del juego.
4. Evidencia del funcionamiento de los tres niveles.
5. Explicación general del juego.
6. Explicación de las cinco funciones implementadas en NASM.
7. Explicación de cómo compilar y ejecutar el proyecto.
8. Captura o evidencia del resumen final del juego.

El repositorio de GitHub deberá contener:

1. Código fuente completo del proyecto.
2. Archivos `.c`, `.h` y `.asm` utilizados.
3. Archivo `build.bat` para compilar el proyecto en Windows.
4. Archivo `README.txt` o `README.md` con instrucciones breves de compilación y ejecución.
5. Historial de commits que muestre avance real del proyecto.

Importante

No se evaluará únicamente el PDF. El enlace al repositorio de GitHub es obligatorio, ya que ahí se revisará el código fuente, el archivo `build.bat`, el archivo `README` y el historial de commits.

Contenido mínimo del reporte

El reporte deberá entregarse en formato PDF y contener al menos las siguientes secciones:

1. Portada.
2. Enlace al repositorio de GitHub.
3. Introducción.
4. Objetivo del proyecto.

5. Descripción general del juego.
6. Explicación de la representación del mapa.
7. Explicación del funcionamiento de la ventana visible de 20 x 20.
8. Explicación de las funciones implementadas en C.
9. Explicación detallada de las cinco funciones implementadas en NASM.
10. Explicación de la fórmula usada para calcular el puntaje.
11. Capturas de pantalla de los tres niveles.
12. Captura del resumen final del juego.
13. Instrucciones de compilación y ejecución.
14. Conclusiones.

Nota

No pegar todo el código fuente dentro del reporte. El código deberá estar disponible en el repositorio de GitHub. En el reporte pueden incluirse fragmentos pequeños, nombres de funciones o variables importantes para explicar la lógica del programa.

Importante

El enlace al repositorio de GitHub debe funcionar correctamente. Si el repositorio es privado, el alumno deberá asegurarse de dar acceso al profesor antes de la fecha de entrega.

Criterios de evaluación

Criterio	Puntos
Juego funcional en consola.	10
Tres niveles funcionales.	8
Uso de matrices de al menos 60 x 60.	8
Implementación de ventana visible de 20 x 20.	15
Movimiento correcto del jugador.	8
Funcionamiento de paredes, llave, puerta y salida.	8
Sistema de monedas y resumen por nivel.	8
Función en NASM para contar caracteres del mapa.	10
Función en NASM para validar movimiento.	8
Función en NASM para calcular puntaje.	5

Función en NASM para detectar objetos en una celda.	4
Función en NASM para contar celdas libres.	3
Reporte PDF con capturas, enlace a GitHub, explicación, README, build.bat e historial de commits.	5
Total	100

Penalizaciones

Se podrán aplicar penalizaciones en los siguientes casos:

Situación	Penalización
No se incluye enlace al repositorio de GitHub dentro del PDF.	Hasta 15 puntos
El repositorio no contiene el código fuente completo.	Hasta 20 puntos
El repositorio no contiene archivo <code>build.bat</code> o <code>README</code> .	Hasta 10 puntos
El repositorio tiene un solo commit o no muestra avance real.	Hasta 15 puntos
El proyecto no compila.	Hasta 40 puntos
El proyecto no ejecuta correctamente.	Hasta 30 puntos
No se usa NASM.	Hasta 30 puntos
Solo se usa NASM de forma decorativa y no funcional.	Hasta 20 puntos
Falta una o más funciones obligatorias en NASM.	Hasta 25 puntos
El total de monedas está escrito manualmente.	Hasta 15 puntos
No hay tres niveles.	Hasta 10 puntos
No se implementa la ventana visible de 20 x 20.	Hasta 15 puntos
Código desordenado o sin comentarios.	Hasta 10 puntos
No se entrega reporte.	Hasta 10 puntos
No se incluyen capturas de ejecución.	Hasta 5 puntos

Restricciones

El proyecto deberá respetar las siguientes restricciones:

- No se permite entregar únicamente capturas o reporte sin enlace al repositorio.
- No se permite entregar un repositorio con un solo commit realizado al final.
- No se permite omitir el archivo `build.bat` O el `README` como sustituto del reporte.
- No se permite entregar un proyecto hecho únicamente en C.
- No se permite entregar un proyecto hecho únicamente en NASM.

- No se permite definir manualmente el total de monedas de cada nivel.
- No se permite usar una matriz menor a la solicitada.
- No se permite mostrar todo el mapa completo si no se implementa la ventana visible.
- No se permite omitir funciones obligatorias de NASM.
- No se permite entregar código que no compile.
- No se permite entregar un proyecto descargado de internet sin modificaciones propias.